EX.NO:9 AZURE LOAD TESTING AND PIPELINE

Aim:
To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment

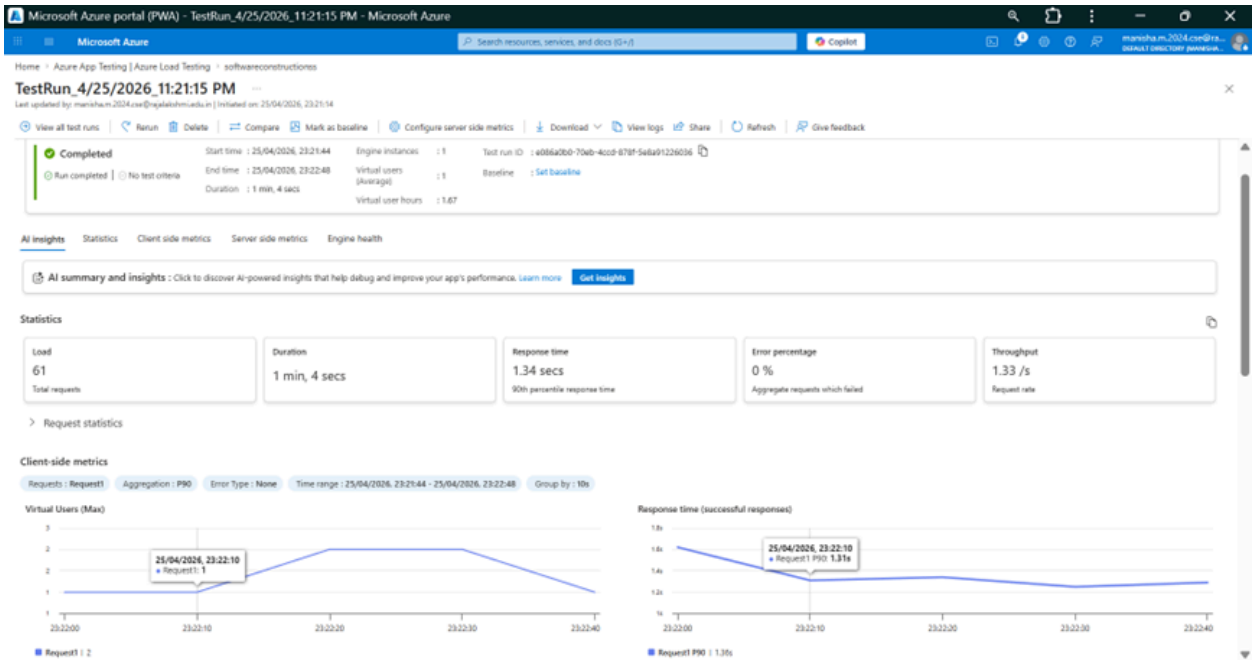
Load Testing Azure Load Testing:
Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

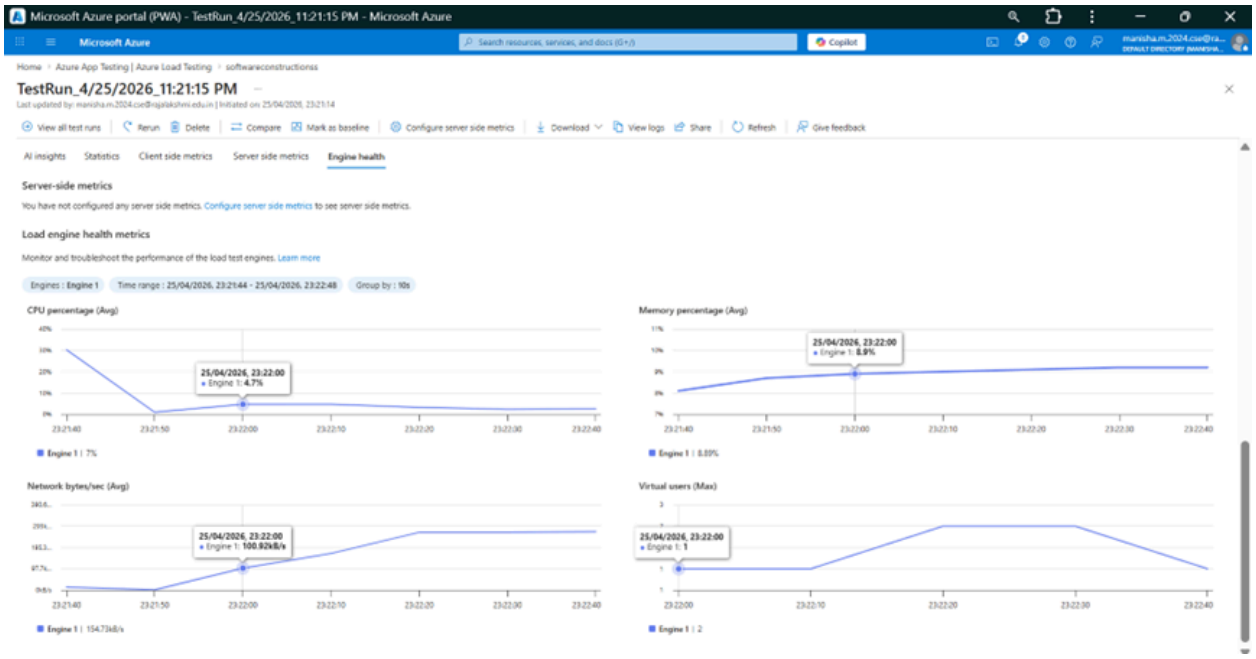
Steps to Create an Azure Load Testing Resource:
Before you run your first test, you need to create the Azure Load Testing resource:
1. Sign in to Azure Portal Go to https://portal.azure.com and log in.
2. Create the Resource o Go to Create a resource → Search for "Azure Load Testing".
   - Select Azure Load Testing and click Create.
3. Fill in the Configuration Details o Subscription: Choose your Azure subscription.
   - Resource Group: Create new or select an existing one.
   - Name: Provide a unique name (no special characters).
   - Location: Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create. 6. Once deployment is complete, click Go to resource.

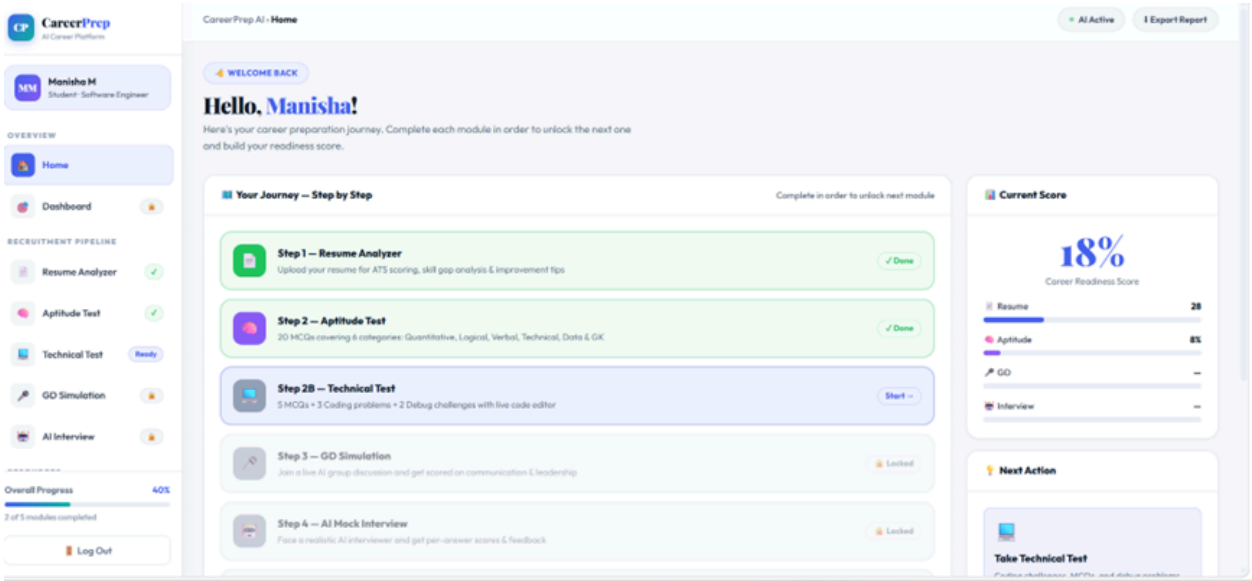
Steps to Create and Run a Load Test:
Once your resource is ready:
1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab
   - Test Name: Provide a unique name.
   - Description: (Optional) Add test purpose.
   - Run After Creation: Keep checked.

3. Load Settings
   - Test URL: Enter the target endpoint (e.g., https://yourapi.com/products).
4. Click Review + Create → Create to start the test

Load Testing







Pipelines – AI Resume Assistant
Description

This experiment demonstrates how to connect a **GitHub-hosted Flask-based AI Resume Assistant project** with Azure DevOps.

The pipeline will:

- Install Python dependencies
- Run basic tests (resume parsing / API checks)
- Validate application build
- Publish artifacts

This ensures that every commit is automatically tested for reliability and smooth deployment.

Steps

1. Connect GitHub to Azure DevOps

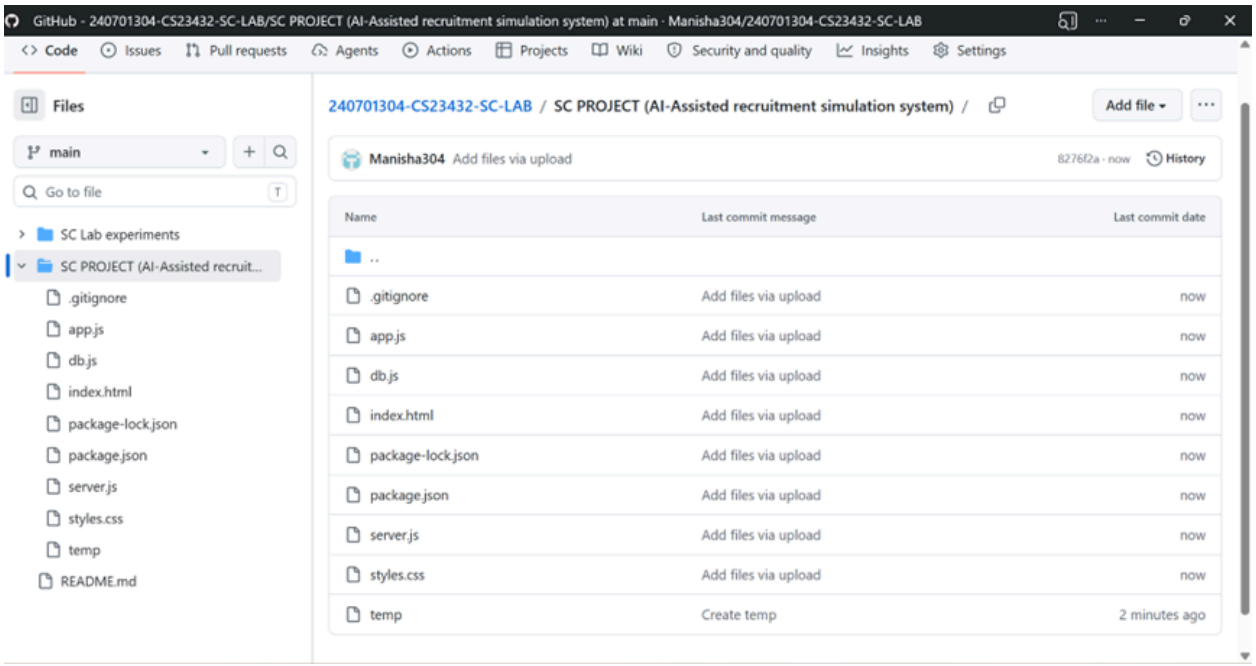
- Go to Azure DevOps → Create a new project
- Click **Pipelines → New Pipeline**
- Select **GitHub** as source
- Authorize GitHub access
- Select your **AI Resume Assistant repository**

2. Create azure-pipelines.yml in Repo Root

Create a file named **azure-pipelines.yml** in your GitHub repo and add the following:

```
trigger:
- main

pool:
  vmImage: ubuntu-latest

steps:
- checkout: self

- task: UseNode@1
  inputs:
    version: '18.x'
  displayName: "Set up Node.js"

- script: |
    cd "SC PROJECT (AI-Assisted recruitment simulation system)"
    npm install --prefer-offline --no-audit --progress=false
  displayName: "Install dependencies"

- script: |
    cd "SC PROJECT (AI-Assisted recruitment simulation system)"
    node -e "console.log('✅ AI-Assisted Recruitment Simulation Pipeline Executed
Successfully!')"
  displayName: "Run Test Script"

- task: PublishBuildArtifacts@1
  inputs:
    PathtoPublish: 'SC PROJECT (AI-Assisted recruitment simulation system)'
    ArtifactName: 'AIRecruitmentArtifact'
    publishLocation: 'Container'
  displayName: "Publish Artifact"
```

Pipeline



Result:

Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.